JAVA PROGRAMMING

Chapter 14: Creating Objects in Java

Personal Notes

In the previous chapter you learned what OOP is and what classes and objects are. This chapter zooms in on one specific thing: HOW objects are actually created in Java, what happens behind the scenes, and how constructors make object setup clean and safe.

Understanding object creation deeply matters because it shows up everywhere — from writing your first class to debugging real-world Java apps. Get this right once, and the rest of OOP becomes much easier.

1. What is an Object? (Quick Recap)

In simple terms, an object is an instance of a class. It bundles together two things:

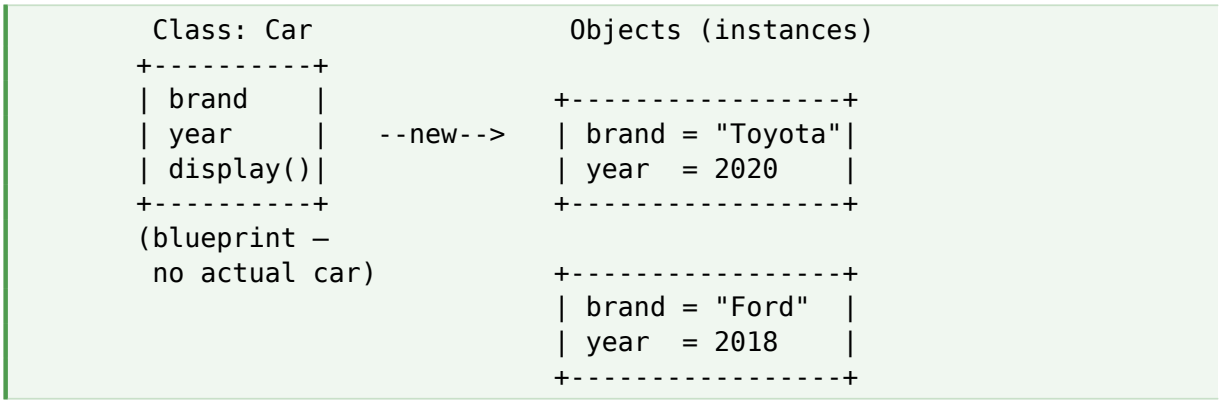
- Data — the variables (fields) that describe its state
- Methods — the functions that define what it can do

Think of it like this:

- A class is a blueprint — the design
- An object is the real-world entity built from that blueprint

Real-World Example

If Car is the class, then a specific car like a Toyota (2020) is an object. The class describes what every car has and can do — the object is one actual car with specific values.



2. Creating Objects: The Two Steps

Creating an object in Java is always a two-step process:

1. Declare a class — define the structure (fields) and behavior (methods)
2. Instantiate the class — create an actual object using the new keyword

That's it. Every object you'll ever create in Java follows this pattern.

3. Step 1: Declaring a Class

A class declaration defines what kind of data the object will hold and what actions it can perform.

Basic Syntax

```
class Car {
    // Fields (data / state)
    String brand;
    int year;

    // Method (behavior)
    void display() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}
```

Notice — the class itself uses NO memory yet. It's just a template. Memory is only allocated when an actual object is created from it.

4. Step 2: Instantiating the Class

Once the class exists, you can create as many objects (instances) from it as you want using the new keyword.

Example

```
public class Main {
    public static void main(String[] args) {

        Car car1 = new Car();          // create first object
        car1.brand = "Toyota";
        car1.year = 2020;
        car1.display();                // Brand: Toyota, Year: 2020

        Car car2 = new Car();          // create second object
        car2.brand = "Ford";
        car2.year = 2018;
        car2.display();                // Brand: Ford, Year: 2018
    }
}
```

Anatomy of an Object Creation Line

```
     Car          car1        =       new Car()       ;
     |             |           |          |
     v             v           v          v
```

```
   Type of      Reference     Assign     Actually
   variable     variable    operator    creates the
   (the class)  (the name              object in heap
                you'll use            via the constructor
                to access)
```

Four things happen in one line: (1) Java declares a reference variable car1 of type Car. (2) The new keyword allocates memory for a new Car object on the heap. (3) The default constructor runs and initializes the fields. (4) The reference (memory address) of the new object is stored in car1.

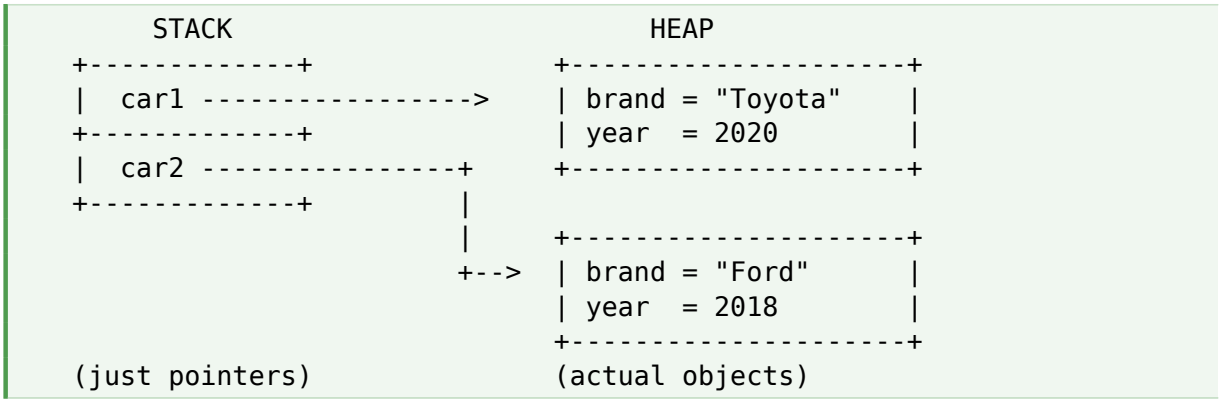
5. Memory: What new Actually Does

Java handles memory automatically, which makes life much easier than in languages like C++. But it helps to understand what's happening behind the scenes.

5.1 Stack vs Heap

- Reference variables (like car1) sit on the STACK — they hold the address of the object
- Actual objects live on the HEAP — that's where new allocates space

Visual



5.2 Garbage Collection

Unlike C++, Java does NOT make you free memory manually:

- All objects are created on the heap
- Java's Garbage Collector (GC) automatically deletes objects that no longer have any references pointing to them
- There is no delete keyword and no manual cleanup

```
Car car = new Car();   // object created on heap
car = null;            // no reference now → GC will reclaim it later
```

> **Why this matters:** Forgetting to free memory in C/C++ causes memory leaks. Java's automatic memory management removes that whole class of bugs — one of the biggest reasons Java is considered safer than C++.

6. Constructors

A constructor is a special method that runs AUTOMATICALLY when you create an object using new. Its job is to set up the object's initial state.

Three Rules of Constructors

- The constructor name MUST exactly match the class name
- Constructors have NO return type — not even void
- They are called automatically by new — you never call them manually

7. Types of Constructors

7.1 Default Constructor

If you don't write any constructor yourself, Java automatically provides a hidden default one. It takes no arguments and does nothing special — just creates the object with default field values.

Example with an explicit default constructor

```
class Car {
    String brand;
    int year;

    // Explicit default constructor
    Car() {
        System.out.println("Default constructor called");
    }
}

// Usage:
Car car = new Car();     // prints: Default constructor called
// car.brand is null, car.year is 0 (defaults)
```

> **Important:** The moment you write ANY constructor (even a parameterized one), Java stops providing the default one for free. If you still want a no-argument constructor, you must write it yourself.

7.2 Parameterized Constructor

A parameterized constructor takes arguments so you can initialize the object with specific values right when you create it.

```
class Car {
```

```
    String brand;
    int year;

    // Parameterized constructor
    Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    void display() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}

// Usage:
Car car1 = new Car("Toyota", 2020);
car1.display();    // Brand: Toyota, Year: 2020

Car car2 = new Car("Ford", 2018);
car2.display();    // Brand: Ford, Year: 2018
```

This is cleaner than manually setting each field one by one. The object is fully ready to use the moment it's created.

7.3 The this Keyword

In the constructor above, you saw lines like this.brand = brand. What does this mean?

- this refers to the CURRENT object being created
- this.brand is the field of the object
- brand (without this) refers to the parameter passed in

Without this, the names would clash. Java would think you're assigning the parameter to itself.

```
Car(String brand, int year) {
    this.brand = brand;   // field = parameter
    this.year  = year;
}
```

8. Constructor Overloading

A class can have multiple constructors as long as their parameter lists are different. This is just method overloading applied to constructors.

```
class Car {
    String brand;
    int year;

    // No-arg constructor
    Car() {
```

```
        this.brand = "Unknown";
        this.year = 0;
    }

    // One-arg constructor
    Car(String brand) {
        this.brand = brand;
        this.year = 2024;
    }

    // Two-arg constructor
    Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }
}

// Usage — pick whichever fits your situation
Car c1 = new Car();
Car c2 = new Car("Tesla");
Car c3 = new Car("BMW", 2022);
```

Tip: Constructor overloading gives users of your class flexibility. They can create an object with full info, partial info, or no info — depending on what they have available.

9. Default Field Values

When an object is created and you don't initialize a field, Java automatically gives it a default value based on its type. This is one of those quiet facts that catches beginners off guard.

Field Type	Default Value	Example
int / byte / short / long	0	int x; → 0
double / float	0.0	double d; → 0.0
boolean	false	boolean b; → false
char	'\u0000'	char c; → empty char
Object reference (String, etc.)	null	String s; → null

Important: These defaults apply ONLY to fields (instance variables). Local variables inside methods do NOT get default values — you must initialize them before use, or the compiler will complain.

10. Constructor Execution Flow

When you call new Car("Toyota", 2020), here's what actually happens in order:

```
  new Car("Toyota", 2020)
       |
       v
  1.  JVM allocates memory on the heap for a Car object
       |
       v
  2.  All fields are set to their default values
       (brand = null, year = 0)
       |
       v
  3.  The matching constructor runs
       (this.brand = "Toyota", this.year = 2020)
       |
       v
  4.  The reference (address) of the new object
       is returned and stored in car1
       |
       v
       Done. Object is ready to use.
```

11. Common Mistakes

Mistake 1: Adding a return type to the constructor

```
void Car() { ... }     // WRONG — this becomes a regular method, not a
constructor
Car()      { ... }     // correct
```

If you add void (or any return type), it's no longer a constructor — Java treats it as a regular method that just happens to share the class name.

Mistake 2: Forgetting this.field = field

```
Car(String brand, int year) {
    brand = brand;   // WRONG — assigns the parameter to itself!
    year  = year;    // The fields stay null and 0
}
```

Always use this.fieldName when the parameter name matches the field name.

Mistake 3: Expecting the default constructor after writing your own

```
class Car {
    Car(String b, int y) { ... }   // only this constructor exists now
}

Car c = new Car();   // ERROR — no no-arg constructor available
```

Mistake 4: Calling a constructor like a regular method

```
Car car = new Car();
car.Car();          // WRONG — constructors are not called manually
car = new Car();    // correct way to reinitialize
```

Mistake 5: Confusing reference and object

Car c1 = c2; does NOT copy the object — it only copies the reference. Both c1 and c2 now point to the SAME object.

```
Car c1 = new Car("BMW", 2022);
Car c2 = c1;          // c2 points to the SAME object as c1
c2.year = 2024;
System.out.println(c1.year);   // 2024 — both see the change
```

12. Best Practices

- Always use constructors instead of setting fields one by one — it's cleaner and prevents "half-initialized" objects
- Follow encapsulation — keep fields private and use getters/setters or constructors to set them
- Provide overloaded constructors for flexibility (no-arg, partial, full)
- Don't create unnecessary objects — reuse where possible (especially for immutable things like Strings)
- Use this.field = field consistently — it makes intent clear
- Validate inputs inside the constructor when needed (e.g., reject negative ages)

13. Quick Reference

Term / Keyword	Meaning
new	Allocates memory and creates an object on the heap
Constructor	Special method that initializes a new object
Default constructor	No-arg constructor; provided automatically if you don't write any
Parameterized	Constructor that takes arguments to set initial values
this	Refers to the current object (used to disambiguate fields)
null	Default for any uninitialized reference field
Garbage Collector	Automatically removes unused objects from memory
Reference variable	Holds the memory address of an object
Heap	Memory region where objects live

Term / Keyword	Meaning
Stack	Memory region where reference variables and locals live

14. Practice Problems

Try these on your own. Each one strengthens a different aspect of object creation.

3. Create a Student class with name, age, and marks. Use a parameterized constructor and create 3 student objects.
4. Create a Book class with title, author, price. Add overloaded constructors: no-arg, title-only, and full.
5. Create a Rectangle class with length and width. Add an area() method and a constructor that validates positive values.
6. Create an Employee class with id, name, salary. Use this keyword properly throughout.
7. Create a class with a static counter that tracks how many objects have been created.
8. Build a BankAccount class with private balance, a constructor for opening balance, and deposit/withdraw methods.
9. Create two Car objects, assign one to another (c2 = c1), modify c2, and observe that c1 also changed. Explain why.
10. Build a Mobile class with brand, model, and price. Use overloaded constructors to allow different ways of creation.

15. Key Points to Remember

- Objects are created using the new keyword
- Each object gets its own copy of instance variables
- Java handles memory automatically — no manual deletion
- Constructors run automatically when an object is created
- If no constructor is defined, Java provides a default one
- Writing any constructor removes the default — write a no-arg one if you still need it
- Use this to refer to the current object inside the class
- Reference variables hold addresses, NOT actual objects

> **Key Takeaway:** Creating objects in Java is simple on the surface — just Class name = new Class(); — but a lot is happening underneath: memory allocation on the heap, default values being set, the constructor running, and the reference being returned. Once you internalize this flow, OOP, inheritance, polymorphism, and even debugging memory issues all become much clearer.

16. What's Next?

Now that object creation is solid, the natural next steps are:

- Constructor chaining — calling one constructor from another using this() and super()
- Static fields and methods — data shared across all objects
- Inheritance — building child classes from parents
- The toString(), equals(), and hashCode() methods every object inherits
- Inner classes and anonymous classes
- Object cloning and copy constructors